

Chapter 14. Attributes

In .NET, you can annotate components, types, and their members with *attributes*. An attribute's purpose is to control or modify the behavior of a framework, a tool, the compiler, or the CLR. For example, in [Chapter 1](#), I showed a class annotated with the `[TestClass]` attribute. This told a unit testing framework that the annotated class contains some tests to be run as part of a test suite.

Attributes are passive containers of information that do nothing on their own. To draw an analogy with the physical world, if you print out a shipping label containing an address and tracking information and attach it to a package, that label will not in itself cause the package to make its way to a destination. Such a label is useful only once the package is in the hands of a shipping company. When the company picks up your parcel, it'll expect to find the label and will use it to work out how to route your package. So the label is important, but ultimately, its only job is to provide information that some system requires. .NET attributes work the same way—they have an effect only if something goes looking for them. Some attributes are handled by the CLR or the compiler, but these are in the minority. The majority of attributes are consumed by frameworks, libraries, tools (such as a unit test runner), or your own code.

Applying Attributes

To avoid having to introduce an extra set of concepts into the type system, .NET models attributes as instances of .NET types. To be used as an attribute, a type must derive from the `System.Attribute` class, but it can otherwise be entirely ordinary. To apply an attribute, you put the type's name in square brackets, and this usually goes directly before the attribute's target. (Since C# mostly ignores whitespace, attributes don't have to be on a separate line, but that is the convention when the target is

a type or a member.) [Example 14-1](#) shows some attributes from Microsoft's test framework. I've applied one to the class to indicate that this contains tests I'd like to run, and I've also applied attributes to individual methods, telling the test framework which ones represent tests and which contain initialization code to be run before each test.

Example 14-1. Attributes in a unit test class

```
using Microsoft.VisualStudio.TestTools.UnitTesting;

namespace ImageManagement.Tests;

[TestClass]
public class WhenPropertiesRetrieved
{
    private ImageMetadataReader? _reader;

    [TestInitialize]
    public void Initialize()
    {
        _reader = new ImageMetadataReader(TestFiles.GetImage());
    }

    [TestMethod]
    public void ReportsCameraMaker()
    {
        Assert.AreEqual(_reader!.CameraManufacturer, "Fabrikam");
    }

    [TestMethod]
    public void ReportsCameraModel()
    {
        Assert.AreEqual(_reader!.CameraModel, "Fabrikam F450D");
    }
}
```

If you look at the documentation for most attributes, you'll find that their real name ends with `Attribute`. If there's no class with the name you specify in the brackets, the C# compiler tries appending `Attribute`, so in [Example 14-1](#) the `[TestClass]` attribute refers to the `TestClassAttribute` class. If you really want to, you can spell the class

name out in full—for example, `[TestClassAttribute]`—but it’s more common to use the shorter version.

If you want to apply multiple attributes, you have two options. You can either provide multiple sets of brackets or put multiple attributes inside a single pair of brackets, separated by commas.

Some attribute types can take constructor arguments. For example, Microsoft’s test framework includes a `TestCategoryAttribute`. When running tests, you can choose to execute only those in a certain category. This attribute requires you to pass the category name as a constructor argument, because there would be no point in applying this attribute without specifying the name. As [Example 14-2](#) shows, the syntax for specifying an attribute’s constructor arguments is unsurprising.

Example 14-2. Attribute with constructor argument

```
[TestCategory("Property Handling")]
[TestMethod]
public void ReportsCameraMaker()
{
    ...
}
```

You can also specify property or field values. Some attributes have features that can be controlled only through properties or fields, and not constructor arguments. (If an attribute has lots of optional settings, it’s usually easier to present these as properties or fields, instead of defining a constructor overload for every conceivable combination of settings.) The syntax for this is to write one or more `PropertyOrFieldName=Value` entries after the constructor arguments (or instead of them, if there are no constructor arguments). [Example 14-3](#) shows another attribute used in unit testing, `ExpectedExceptionAttribute`, which allows you to specify that when your test runs, you expect it to throw a particular exception. The exception type is mandatory, so we pass that as a constructor argument, but this attribute also allows you to state whether the test runner should accept exceptions of a type derived from the one specified. (By default, it will accept only an exact match.) This is controlled with the `AllowDerivedTypes` property.

Example 14-3. Specifying optional attribute settings with properties

```
[ExpectedException(typeof(ArgumentException), AllowDerivedTypes = true)]  
[TestMethod]  
public void ThrowsWhenNameMalformed()  
{  
    ...  
}
```

Applying an attribute will not cause it to be constructed. All you are doing when you apply an attribute is providing instructions on how the attribute should be created and initialized if something should ask to see it. (There is a common misconception that method attributes are instantiated when the method runs. Not so.) When the compiler builds the metadata for an assembly, it includes information about which attributes have been applied to which items, including a list of constructor arguments and property values, and the CLR will dig that information out and use it only if something asks for it. For example, when you tell Visual Studio to run your unit tests, it will load your test assembly, and then for each public type, it asks the CLR for any test-related attributes. That's the point at which the attributes get constructed. If you were simply to load the assembly by, say, adding a reference to it from another project and then using some of the types it contains, the attributes would never come into existence—they would remain as nothing more than a set of building instructions frozen into your assembly's metadata. In some cases they might not even make it that far. If you're using Native AOT in conjunction with the JSON serialization mechanisms shown in [Chapter 15](#), the relevant attributes are processed during compilation and get trimmed out of the final output.

Attribute Targets

Attributes can be applied to numerous different kinds of targets. You can put attributes on any of the features of the type system represented in the reflection API that I showed in [Chapter 13](#). Specifically, you can apply attributes to assemblies, modules, types, methods, method parameters, constructors, fields, properties, events, and generic type parameters. In addition, you can supply attributes that target a method's return value.

For most of these, you denote the target simply by putting the attribute in front of it. But that's not an option for assemblies or modules, because there is no single feature that represents those in your source code—everything in your project goes into the assembly it produces, and modules are likewise an aggregate. So for these, we have to state the target explicitly at the start of the attribute. The specific assembly-level attribute shown in [Example 14-4](#) will often be found in a *GlobalSuppressions.cs* file. Visual Studio sometimes makes suggestions for modifying your code, and if you want to suppress these, one option is to use assembly-level attributes.

Example 14-4. Assembly-level attributes

```
[assembly: System.Diagnostics.CodeAnalysis.SuppressMessage(
    "Style",
    "IDE0060:Remove unused parameter",
    Justification = "This is just some example code from a book",
    Scope = "member",
    Target = "~M:Idg.Examples.SomeMethod")]
```

You can put assembly-level attributes in any file. The sole restriction is that they must appear before any namespace or type definitions. The only things that should come before assembly-level attributes are whichever `using` directives you need, comments, and whitespace (all of which are optional).

Module-level attributes follow the same pattern, although they are much less common, not least because multimodule assemblies are pretty rare and are not supported in the latest versions of .NET—they only work on .NET Framework. [Example 14-5](#) shows how to configure the debuggability of a particular module, should you want one module in a multimodule assembly to be easily debuggable but the rest to be JIT-compiled with full optimizations. (This is a contrived scenario so that I can show the syntax. In practice, you're unlikely ever to want to do this.) I'll talk about the `DebuggableAttribute` later, in ["JIT compilation"](#).

Example 14-5. Module-level attribute

```
using System.Diagnostics;

[module: Debuggable(DebuggableAttribute.DebuggingModes.DisableOptimizations)]
```

Another kind of target that needs qualification is a compiler-generated field. You get these with properties in which you do not supply code for the getter or setter, and also in `event` members without explicit `add` and `remove` implementations. The attributes in [Example 14-6](#) apply to the fields that hold the property's value and the delegate for the event; without the `field:` qualifiers, attributes in those positions would apply to the property or event itself.

Example 14-6. Attribute for compiler-generated property and event fields

```
[field: NonSerialized]
public int DynamicId { get; set; }

[field: NonSerialized]
public event EventHandler? Frazzled;
```

Methods' return values can be annotated, and this also requires qualification, because return value attributes go in front of the method, the same place as attributes that apply to the method itself. (Attributes for parameters do not need qualification, because these appear inside the parentheses with the parameters.) [Example 14-7](#) shows a method with attributes applied to both the method and the return type. (The attributes in this example are part of the interop services that enable .NET code to call external code, such as OS APIs. This example imports a function from a Win32 DLL, enabling you to use it from C#. There are several different representations for Boolean values in unmanaged code, so I've annotated the return type here with a `MarshalAsAttribute` to say which particular one the CLR should expect.)

Example 14-7. Method and return value attributes

```
[LibraryImport("User32.dll")]  
[return: MarshalAs(UnmanagedType.Bool)]  
internal static partial bool IsWindowVisible(IntPtr hWnd);
```

C# 11.0 added a new feature for attributes that target either return values or arguments. Sometimes, attributes describe relationships between multiple parameters, or a relationship between a method's return value and a parameter. An attribute has just one target, so we refer to related arguments by name. [Example 14-8](#) shows an example from the .NET runtime library: the `string.Create` method. (This method enables you to change localization and other format settings when using an interpolated string. You can write `string.Create(CultureInfo.InvariantCulture, $"Value: {v}")`, for example.) It uses the `InterpolatedStringHandlerArgumentAttribute` to tell the compiler that it wants to provide custom string interpolation handling, and that this will be managed by the `DefaultInterpolatedStringHandler` type. The compiler needs to know whether any of the method's other arguments are involved with the string interpolation, and this method has indicated that the `provider` argument needs to be passed as a constructor argument to `DefaultInterpolatedStringHandler`.

Example 14-8. Referring to a method argument by name in an attribute before C# 11.0

```
public static string Create(  
    IFormatProvider? provider,  
    [InterpolatedStringHandlerArgument("provider")]  
    ref DefaultInterpolatedStringHandler handler)
```

Referring to arguments by name in a string was always slightly unsatisfactory. It's easy to mistype the name, and it is hard for refactoring tools to process these correctly—if you rename the argument, that string also needs to change. This is exactly the scenario that `nameof` is designed for, but you couldn't use it here because parameters were in scope only inside the method's body. C# 11.0 relaxed the scoping rules very slightly: `nameof` is now allowed to refer to argument names in attributes applied to the

method's arguments or return value, so if you look at the source for `string.Create` you'll find it now looks like [Example 14-9](#).

Example 14-9. Referring to a method argument by name in an attribute using `nameof`

```
public static string Create(  
    IFormatProvider? provider,  
    [InterpolatedStringHandlerArgument(nameof(provider))]  
    ref DefaultInterpolatedStringHandler handler)
```

How are we to apply method attributes in cases where we don't write the method declaration explicitly? As you saw in [Chapter 9](#), the lambda syntax lets us write an expression whose value is a delegate. The compiler generates a normal method to hold the code (typically in a hidden class), and we might want to pass that method to a framework that uses attributes to control its functionality, such as the ASP.NET Core web framework. [Example 14-10](#) shows how we can specify these attributes when using a lambda.

Example 14-10. Lambda with attributes

```
app.MapGet(  
    "/items/{id}",  
    [Authorize] ([FromRoute] int id) => $"Item {id} requested");
```

The `MapGet` method here tells the ASP.NET Core framework how our application should behave when it receives `GET` requests on URLs matching a particular pattern. The first argument specifies the pattern, and the second is a delegate that defines the behavior. I've used the lambda syntax here, and I've applied a couple of attributes.

The first attribute is `[Authorize]`. This appears before the parameter list, so its target is the whole method. (You can also use a `return:` attribute in this position.) This causes ASP.NET Core to block unauthenticated requests that match this URL pattern. The `[FromRoute]` attribute is inside the parameter list's parentheses, so it applies to the `id` parameter, and it tells ASP.NET Core that we want that particular parameter's value to be taken from the expression of the same name in the URL pattern. So

if a request came in for `https://myserver/items/42`, ASP.NET Core would first check that the request meets the application's configured requirements for authentication and authorization, and if so, it would then invoke my lambda passing `42` as the `id` argument.

NOTE

[Example 9-23](#) in [Chapter 9](#) showed that you can omit details in certain cases. The parentheses around the parameter list are normally optional for 1-argument lambdas. However, the parentheses *must* be present if you apply attributes to a lambda. To see why, imagine [Example 14-10](#) without parentheses around the parameter list: it would be unclear whether the attributes were meant to apply to the method or the parameter.

Compiler-Handled Attributes

The C# compiler recognizes certain attribute types and handles them in special ways. For example, assembly names and versions are set via attributes and also some related information about your assembly. As [Chapter 12](#) described, the build process generates a hidden source file containing these for you. If you're curious, it usually ends up in the `obj\Debug` or `obj\Release` folder of your project, and it will be named something like `YourProject.AssemblyInfo.cs`. [Example 14-11](#) shows a typical example.

Example 14-11. A typical generated file with assembly-level attributes

```
//-----  
// <auto-generated>  
//     This code was generated by a tool.  
//     Runtime Version:4.0.30319.42000  
//  
//     Changes to this file may cause incorrect behavior and will be lost if  
//     the code is regenerated.  
// </auto-generated>  
//-----  
  
using System;  
using System.Reflection;
```

```
[assembly: System.Reflection.AssemblyCompanyAttribute("MyCompany")]
[assembly: System.Reflection.AssemblyConfigurationAttribute("Debug")]
[assembly: System.Reflection.AssemblyFileVersionAttribute("1.0.0.0")]
[assembly: System.Reflection.AssemblyInformationalVersionAttribute("1.0.0")]
[assembly: System.Reflection.AssemblyProductAttribute("MyApp")]
[assembly: System.Reflection.AssemblyTitleAttribute("MyApp")]
[assembly: System.Reflection.AssemblyVersionAttribute("1.0.0.0")]

// Generated by the MSBuild WriteCodeFragment class.
```

Old versions of the .NET Framework SDK did not generate this file at build time, so if you work on older projects, you will often find these attributes in a file called *AssemblyInfo.cs*. (By default Visual Studio hid this inside the project's Properties node in Solution Explorer, but it was still just an ordinary source file.) The advantage of the file generation used in modern projects is that names are less likely to drift out of sync. For example, by default the assembly Product and Title will be the same as the project filename. If you rename the project file, the generated *YourRenamedProject.AssemblyInfo.cs* will change to match (unless you added `<Product>` and `<AssemblyTitle>` properties to your project file, in which case it will use those), whereas with the old *AssemblyInfo.cs* approach you could accidentally end up with mismatched names. Similarly, if you build a NuGet package from your project, certain properties end up in both the NuGet package and the compiled assembly. When these are all generated from information in the project file, it's easier to keep things consistent.

Even though you only control these attributes indirectly, it's useful to understand them since they affect the compiler output.

Names and versions

As you saw in [Chapter 12](#), assemblies have a compound name. The simple name, which is typically the same as the filename but without the `.exe` or `.dll` extension, is configured as part of the project settings. The name also includes a version number, and this is controlled with an attribute, as [Example 14-12](#) shows.

Example 14-12. Version attributes

```
[assembly: AssemblyVersion("1.0.0.0")]  
[assembly: AssemblyFileVersion("1.0.0.0")]
```

As you may recall from [Chapter 12](#), the first of these sets the version part of the assembly's name. The second has nothing to do with .NET—the compiler uses this to generate a Win32-style version resource. This is the version number end users will see if they select your assembly in Windows Explorer and open the Properties window.

The culture is also part of the assembly name. This will often be set automatically if you're using the satellite resource assembly mechanisms described in [Chapter 12](#). You can set it explicitly with the `AssemblyCulture` attribute, but for nonresource assemblies, the culture should usually not be set. (The only culture-related assembly-level attribute you will normally specify explicitly is the `NeutralResourcesLanguageAttribute`, which I showed in [Chapter 12](#).)

Strongly named assemblies have an additional component in their name: the public key token. The easiest way to set up a strong name in Visual Studio is with the “Strong naming” section of your project's properties page (which is inside the Build section). If you're using VS Code or some other editor, you can just add two properties to your `.csproj` file:

`SignAssembly` set to `True`, and `AssemblyOriginatorKeyFile` with the path to your key file. However, you can also manage strong naming from the source code, because the compiler recognizes some special attributes for this. `AssemblyKeyFileAttribute` takes the name of a file that contains a key. Alternatively, you can install a key in the computer's key store (which is part of the Windows cryptography system). If you want to do that, you can use the `AssemblyKeyNameAttribute` instead. The presence of either of these attributes causes the compiler to embed the public key in the assembly and include a hash of that key as the public key token of the strong name. If the key file includes the private key, the compiler will sign your assembly too. If it does not, it will fail to compile, unless you also enable either delay signing or public signing. You can enable delay signing by applying the `AssemblyDelaySignAttribute` with a constructor argument of `true`. Alternatively, you can add either

`<DelaySign>true</DelaySign>` or `<PublicSign>true</PublicSign>` to your `.csproj` file.

WARNING

Although the key-related attributes trigger special handling from the compiler, it still embeds them in the metadata as normal attributes. So, if you use the `AssemblyKeyFileAttribute`, the path to your key file will be visible in the final compiled output. This is not necessarily a problem, but you might prefer not to advertise these sorts of details, so it may be better to use the project-level configuration for strong names than the attribute-based approach.

Description and related resources

The version resource produced by the `AssemblyFileVersion` attribute is not the only information that the C# compiler can embed in Win32-style resources. There are several other attributes providing copyright information and other descriptive text. [Example 14-13](#) shows a typical selection.

Example 14-13. Typical assembly description attributes

```
[assembly: AssemblyTitle("ExamplePlugin")]
[assembly: AssemblyDescription("An example plug-in DLL")]
[assembly: AssemblyConfiguration("Retail")]
[assembly: AssemblyCompany("Endjin Ltd.")]
[assembly: AssemblyProduct("ExamplePlugin")]
[assembly: AssemblyCopyright("Copyright © 2024 Endjin Ltd.")]
[assembly: AssemblyTrademark("")]
```

As with the file version, these are all visible in the Details tab of the Properties window that Windows Explorer can show for the file. And with all of these attributes, you can cause them to be generated by editing the project file.

Caller information attributes

There are some compiler-handled attributes designed for scenarios where your methods need information about the context from which

they were invoked. This is useful for certain diagnostic logging or error handling scenarios, and it is also helpful when implementing a particular interface commonly used in UI code.

[Example 14-14](#) illustrates how you can use these attributes in logging code. If you annotate method parameters with any of these three attributes, the compiler provides some special handling when callers omit the arguments. We can ask for the name of the member (method or property) that called the attributed method, the filename containing the code that called the method, or the line number from which the call was made. [Example 14-14](#) asks for all three, but you can be more selective.

NOTE

These attributes are allowed only for optional parameters. Optional arguments are required to specify a default value, but C# will always substitute a different value when these attributes are present, so the default you specify will not be used if you invoke the method from C# (or F# or Visual Basic, which also support these attributes). Nonetheless, you must provide a default because without one, the parameter is not optional, so we normally use empty strings, `null`, or the number `0`.

Example 14-14. Applying caller info attributes to method parameters

```
public static void Log(  
    string message,  
    [CallerMemberName] string callingMethod = "",  
    [CallerFilePath] string callingFile = "",  
    [CallerLineNumber] int callingLineNumber = 0)  
{  
    Console.WriteLine("Message {0}, called from {1} in file '{2}', line {3}",  
        message, callingMethod, callingFile, callingLineNumber);  
}
```

If you supply all arguments when invoking this method, nothing unusual happens. But if you omit any of the optional arguments, C# will generate code that provides information about the site from which the method was invoked. The default values for the three optional arguments in [Example 14-14](#) will be the name of the method or property that called this

`Log` method, the full path of the source code containing the code that made the call, and the line number from which `Log` was called.

The `CallerMemberName` attribute has a superficial resemblance to the `nameof` operator, which we saw in [Chapter 8](#). Both cause the compiler to create a string containing the name of some feature of the code, but they work quite differently. With `nameof`, you always know exactly what string you'll get, because it's determined by the expression you supply. (E.g., if we were to write `nameof(message)` inside `Log` in [Example 14-14](#), it would always evaluate to `"message"`.) But `CallerMemberName` changes the way the compiler invokes the method to which they apply—`calling Method` has that attribute, and its value is not fixed. It will depend on where this method is called from.

NOTE

You can discover the calling method another way: the `StackTrace` and `StackFrame` classes in the `System.Diagnostics` namespace can report information about methods above you in the call stack. However, these have a considerably higher runtime cost—the caller information attributes calculate the values at compile time, making the runtime overhead very low. (Likewise with `nameof`.) Also, `StackFrame` can determine the filename and line number only if debug symbols are available.

Although diagnostic logging is an obvious application for this, I also mentioned a certain scenario that most .NET UI developers will be familiar with. The runtime libraries define an interface called `INotifyPropertyChanged`. As [Example 14-15](#) shows, this is a very simple interface with just one member, an event called `PropertyChanged`.

Example 14-15. `INotifyPropertyChanged`

```
public interface INotifyPropertyChanged
{
    event PropertyChangedEventHandler? PropertyChanged;
}
```

Types that implement this interface raise the `PropertyChanged` event every time one of their properties changes. The

`PropertyChangedEventArgs` provides a string containing the name of the property that just changed. These change notifications are useful in UIs, because they enable an object to be used with databinding technologies (such as those provided by .NET's WPF UI framework) that can automatically update the UI any time a property changes. Databinding can help you to achieve a clean separation between the code that deals directly with UI types and code that contains the logic that decides how the application should respond to user input.

Implementing `INotifyPropertyChanged` can be both tedious and error-prone. Because the `PropertyChanged` event indicates which property changed as a string, it is very easy to mistype the property name, or to accidentally use the wrong name if you copy and paste the implementation from one property to another. Also, if you rename a property, it's easy to forget to change the text used for the event, meaning that code that was previously correct will now provide the wrong name when raising the `PropertyChanged` event. The `nameof` operator helps avoid mistyping, and helps with renames, but can't always detect cut-and-paste errors. (It won't notice if you fail to update the name when pasting code between properties of the same class, for example.)

Caller information attributes can help make implementing this interface much less error-prone. [Example 14-16](#) shows a base class that implements `INotifyPropertyChanged`, supplying a helper for raising change notifications in a way that exploits one of these attributes. (It uses the null-conditional `?.` operator to ensure that it only invokes the event's delegate if it is non-null. By the way, when you use the operator this way, C# generates code that only evaluates the delegate's `Invoke` method's arguments if it is non-null. So not only does it skip the call to `Invoke` when the delegate is null, it will also avoid constructing the `PropertyChangedEventArgs` that would have been passed as an argument.) This code also detects whether the value really has changed, only raising the event when that's the case, and its return value indicates whether it changed, in case callers might find that useful.

Example 14-16. A reusable `INotifyPropertyChanged` implementation

```
public class NotifyPropertyChanged : INotifyPropertyChanged
{
    public event PropertyChangedEventHandler? PropertyChanged;

    protected bool SetProperty<T>(
        ref T field,
        T value,
        [CallerMemberName] string propertyName = "")
    {
        if (Equals(field, value))
        {
            return false;
        }

        field = value;

        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
        return true;
    }
}
```

The presence of the `[CallerMemberName]` attribute means that a class deriving from this type does not need to specify the property name if it calls `SetProperty` from inside a property setter, as [Example 14-17](#) shows.

Example 14-17. Raising a property changed event

```
public class MyViewModel : NotifyPropertyChanged
{
    private string? _name;

    public string? Name
    {
        get => _name;
        set => SetProperty(ref _name, value);
    }
}
```

Even with the new attribute, implementing `INotifyPropertyChanged` is clearly more effort than an automatic property, where you just write {

get; set; } and let the compiler do the work for you. But it's only a little more complex than an explicit implementation of a trivial field-backed property, and it's simpler than would be possible without `[CallerMemberName]`, because I've been able to omit the property name when asking the base class to raise the event. More importantly, it's less error prone: I can now be confident that the right name will be used every time, even if I rename the property at some point in the future.

There's a fourth caller information attribute,

`CallerArgumentExpression`, that is a little different: instead of telling us where the call came from, it tells us something about what the call looked like. [Example 14-18](#) shows an excerpt from the runtime libraries' `ArgumentNullException` class. It declares a `ThrowIfNull` method that uses this attribute.

Example 14-18. The `CallerArgumentExpressionAttribute` in `ArgumentNullException.ThrowIfNull`

```
public class ArgumentNullException
{
    public static void ThrowIfNull(
        [NotNull] object? argument,
        [CallerArgumentExpression(nameof(argument))] string? paramName = null)
    {
        ...
    }
}
```

As you can see, the `CallerArgumentExpression` attribute takes a single string argument. This must be the name of another parameter in the same method—in this case there is only one other parameter, called `argument`, so it has to refer to that. The effect is that if you call this method without providing a value for the annotated `paramName` argument, the C# compiler will pass a string containing the exact expression you used for the argument that the attribute identified. [Example 14-19](#) shows how this `ThrowIfNull` method is typically called.

Example 14-19. Calling a method that uses

`CallerArgumentExpressionAttribute`

```
static void Greet(string greetingRecipient)
{
    ArgumentNullException.ThrowIfNull(greetingRecipient);
    Console.WriteLine($"Hello, {greetingRecipient}");
}

Greet("world");
Greet(null!);
```

The `Greet` method needs `greetingRecipient` not to be null, so it calls `ArgumentNullException.ThrowIfNull`, passing in `greetingRecipient`. Because this code does not provide a second argument to `ThrowIfNull`, the compiler will provide the full text of the expression we used for the first argument. In this case, that's `"greetingRecipient"`. So the effect is that when I run this program, it throws an `ArgumentNullException` with this message:

```
Value cannot be null. (Parameter 'greetingRecipient')
```

One of the scenarios this attribute supports is to improve assertion messages. For example, unit test libraries typically provide mechanisms for asserting that certain conditions are true after exercising the code under test. The idea is that if your test contains code such as

```
Assert.IsTrue(answer == 42);
```

 the test library could use `[CallerArgumentExpression]` to be able to report the exact expression `(answer == 42)` on failure.

You might expect the `Debug.Assert` method in the runtime libraries to use this for similar reasons, but it does not, because a single-argument overload of that method existed for years before this language feature was added. A two-argument overload using `CallerArgumentExpressionAttribute` on the second argument would be a viable candidate for code such as `Debug.Assert(status == 42);` but that also matches the single-argument method. When a method call could resolve either to a method that is an exact match, or one where the compiler has to generate code to supply missing optional arguments, it pre-

fers the exact match. The only way to use `CallerArgumentExpressionAttribute` would be to remove the existing single-argument method, and this would not be a binary-compatible change. The .NET runtime libraries use this attribute only on exception helper methods. At the time of writing this, none of the widely used .NET testing frameworks use this on their assertion methods, nor do the popular assertion libraries, for similar backward compatibility reasons. The language feature is still relatively new, so perhaps this will change before long.

CLR-Handled Attributes

Some attributes get special treatment at runtime from the CLR. There is no official comprehensive list of such attributes, so in the next few sections, I will just describe some of the most widely used examples.

InternalsVisibleToAttribute

You can apply the `InternalsVisibleToAttribute` to an assembly to declare that any `internal` types or members it defines should be visible to one or more other assemblies. A popular use for this is to enable unit testing of internal types. As [Example 14-20](#) shows, you pass the name of the assembly as a constructor argument.

NOTE

Strong naming complicates matters. Strongly named assemblies cannot make their internals visible to assemblies that are not strongly named, and strongly named assemblies can only reference other strongly named assemblies. When a strongly named assembly makes its internals visible to another strongly named assembly, it must specify not just the simple name but also the public key of the assembly to which it is granting access. And this is not just the public key token I described in [Chapter 12](#)—it is the hexadecimal for the entire public key, which will be several hundred digits. You can discover an assembly's full public key with the .NET SDK's `sn.exe` utility, using the `-Tp` switch followed by the assembly's path.

Example 14-20. InternalsVisibleToAttribute

```
[assembly:InternalsVisibleTo("ImageManagement.Tests")]  
[assembly:InternalsVisibleTo("ImageServices.Tests")]
```

This shows that you can make the types visible to multiple assemblies by applying the attribute multiple times, with a different assembly name each time.

The CLR is responsible for enforcing accessibility rules. Normally, if you try to use an internal class from another assembly, you'll get an error at runtime. (C# won't even let you compile such code, but it's possible to trick the compiler. Or you could write directly in IL. The IL assembler, *ILASM*, does what you tell it and imposes far fewer restrictions than C#. Once you get past the compile-time restrictions, then you'll hit the runtime ones.) But when this attribute is present, the CLR relaxes its rules for the assemblies you list. The compiler also understands this attribute and lets code that tries to use externally defined internal types compile as long as the external library names your assembly in an `InternalsVisibleToAttribute`.

Besides being useful in unit test scenarios, this attribute can also be helpful if you want to split code across multiple assemblies. If you have written a large class library, you might not want to put it into one massive DLL. If it has several areas that your customers might want to use in isolation, it could make sense to split it up so that they can deploy just the parts that they need. However, although you may be able to partition your library's public-facing API, the implementation might not be as easy to divide, particularly if your codebase performs a lot of reuse. You might have many classes that are not designed for public consumption but that you use throughout your code.

If it weren't for the `InternalsVisibleToAttribute`, it would be awkward to reuse shared implementation details across assemblies. Either each assembly would need to contain its own copy of the relevant classes, or you'd need to make them public types in some common assembly. The problem with that second technique is that making types public effectively invites people to use them. Your documentation might state that the

types are for the internal use of your framework and should not be used, but that won't stop some people.

Fortunately, you don't have to make them `public`. Any types that are just implementation details can remain `internal`, and you can make them available to all of your assemblies with the `InternalsVisibleToAttribute` while keeping them inaccessible to everyone else.

JIT compilation

There are a few attributes that influence how the JIT compiler generates code. You can apply the `MethodImplAttribute` to a method, passing values from the `MethodImplOptions` enumeration. Its `NoInlining` value ensures that whenever your method is called by another method, it will be a full method call. Without this, the JIT compiler will sometimes just copy a method's code directly into the calling code.

In general, you'll want to leave inlining enabled. The JIT compiler inlines only small methods, and it's particularly important for tiny methods, such as property accessors. For simple field-based properties, invoking accessors with a normal function call often requires more code than inlining, so this optimization can produce code that's smaller, as well as faster. (Even if the code is not smaller, it may still be faster, because function calls can be surprisingly expensive. Modern CPUs tend to handle long sequential streams of instructions more efficiently than code that leaps around from one location to another.) However, inlining is an optimization with observable side effects—an inlined method does not get its own stack frame. Earlier, I mentioned some diagnostic APIs you can use to inspect the stack, and inlining will change the number of reported stack frames. If you just want to ask the question “Which method is calling me?” the caller info attributes described earlier provide a more efficient way to discover this and will not be defeated by inlining, but if you have code that inspects the stack for any reason, it can sometimes be confused by inlining. So, just occasionally, it's useful to disable it.

Conversely, you can specify `AggressiveInlining`, which encourages the JIT compiler to inline things it might otherwise leave as normal method calls. If you have identified a particular method as being highly perfor-

mance sensitive, it might be worth trying this setting to see if it makes any difference, although be aware that it could make code either slower or faster—it will depend on the circumstances. Conversely, you can disable all optimizations with the `NoOptimization` option (although the documentation implies that this is more for the benefit of the CLR team at Microsoft than for consumers, because it is for “debugging possible code generation problems”).

Another attribute that has an impact on optimization is the `DebuggableAttribute`. The C# compiler automatically applies this to your assembly in Debug builds. The attribute tells the CLR to be less aggressive about certain optimizations, particularly ones that affect variable lifetime and ones that change the order in which code executes. Normally, the compiler is free to change such things as long as the final result of the code is the same, but this can cause confusion if you break into the middle of an optimized method with the debugger. This attribute ensures that variable values and the flow of execution are easy to follow in that scenario.

STAThread and MTAThread

Applications that run only on Windows and that present a UI (e.g., anything using .NET’s WPF or Windows Forms frameworks) typically have the `[STAThread]` attribute on their `Main` method (although you won’t always see it, because the entry point is often generated by the build system for these kinds of applications). This is an instruction to the CLR’s interop services for the Component Object Model (COM), but it has broader implications: you need this attribute on `Main` if you want your main thread to host UI elements.

Various Windows UI features rely on COM under the covers. The clipboard uses it, for example, as do certain kinds of controls. COM has several threading models, and only one of them is compatible with UI threads. One of the main reasons for this is that UI elements have thread affinity, so COM needs to ensure that it does certain work on the right thread. Also, if a UI thread doesn’t regularly check for messages and handle them, deadlock can ensue. If you don’t tell COM that a particular

thread is a UI thread, it will omit these checks, and you will encounter problems.

NOTE

Even if you're not writing UI code, some interop scenarios need the `[STAThread]` attribute, because certain COM components are incapable of working without it. However, UI work is the most common reason for seeing it.

Since COM is managed for you by the CLR, the CLR needs to know that it should tell COM that a particular thread needs to be handled as a UI thread. When you create a new thread explicitly using the techniques shown in [Chapter 16](#), you can configure its COM threading mode, but the main thread is a special case—the CLR creates it for you when your application starts, and by the time your code runs, it's too late to configure the thread. Placing the `[STAThread]` attribute on the `Main` method tells the CLR that your main thread should be initialized for UI-compatible COM behavior.

STA is short for *single-threaded apartment*. Threads that participate in COM always belong to either an STA or a *multithreaded apartment* (MTA). There are other kinds of apartments, but threads have only temporary membership in those; when a thread starts using COM, it must pick either STA or MTA mode. So there is, unsurprisingly, also an `[MTAThread]` attribute.

Debugging Attributes

Some attributes have no effect on normal execution of code, but can change behavior during debugging. If you annotate a method with the `DebuggerStepThroughAttribute`, then by default debuggers will allow that code to run without stopping when you single-step through your code. Some code generation tools will apply this attribute so that the code they generate does not distract you while you are debugging.

If you apply the `DebuggerDisplayAttribute` to a class or struct, it determines how debuggers display instances of that type. For example, if you have written a `Coordinate` type with `X` and `Y` properties, you could an-

notate it with `[DebuggerDisplay("{X}, {Y}")]`. This would tell the debugger to show these property values any time it displays the value of a `Coordinate`.

Build-Time Attributes

Some attributes are handled as part of the build process. NuGet packages can extend the build process in various ways, and they may supply source generators that can add extra code into your project. These are often controlled by attributes. An advantage of this technique is that it works well with trimming because the attributes are processed entirely during the build, instead of using reflection at runtime.

JSON serialization without reflection

The JSON serialization mechanisms described in [Chapter 15](#) use attributes to enable developers to control how .NET objects are represented in JSON. If you use these APIs in the most straightforward way, they will use reflection to discover these attributes at runtime. However, the .NET SDK includes a code generator that can remove any need to use reflection by processing the attributes and other type information at compile time, and generating code. Examples [15-17](#) and [15-18](#) show how to use this, so I won't repeat it here.

Regular expression source generation

The .NET runtime libraries have always offered regular expression support, but .NET 7 has introduced a new way to use these. As [Example 14-21](#) shows, you can annotate a partial method with the `GeneratedRegex` attribute. This triggers a source generator built into the .NET SDK to generate C# code that evaluates the specified regular expression.

Example 14-21. Regular expression source generation

```
public partial class RegexSourceGeneration
{
    [GeneratedRegex(@"[-+]?\\b\\d+\\b")]
    private static partial Regex IsSignedInteger();
}
```

```

    public static bool TextIsSignedInteger(string text)
        => IsSignedInteger().IsMatch(text);
}

```

This enables all the work of parsing the regular expression and determining how best to process it to be done at build time. This improves both startup time and throughput.

Interop

The CLR's interop services define numerous attributes. Since the attributes make sense only in the context of the mechanisms they support, and because there are so many, I will not describe them in full, but [Example 14-22](#) illustrates the kinds of things they can do.

Example 14-22. Interop attributes

```

[LibraryImport("advapi32.dll", EntryPoint = "LookupPrivilegeValueW",
    SetLastError = true, StringMarshalling = StringMarshalling.Utf16)]
[return: MarshalAs(UnmanagedType.Bool)]
internal static partial bool LookupPrivilegeValue(
    [MarshalAs(UnmanagedType.LPWStr)] string? lpSystemName,
    [MarshalAs(UnmanagedType.LPWStr)] string lpName,
    out LUID lpLuid);

```

This uses two interop attributes that we saw earlier in [Example 14-7](#) but in a somewhat more complex way. This calls into a function exposed by *advapi32.dll*, part of the Win32 API. The first argument to the `LibraryImport` attribute tells us that, but unlike the earlier example, this goes on to provide the interop layer with additional information. The `EntryPoint` property deals with the fact that Win32 APIs taking strings sometimes come in two forms, working with either 8-bit or 16-bit characters (some old versions of Windows only supported 8-bit text, to conserve memory) and that we want to call the *Wide* form (hence the `W` suffix). This particular API uses a common Win32 idiom: it returns a Boolean value to indicate success or failure, but it also uses the Windows `SetLastError` API to provide more information in the failure case. The attribute's `SetLastError` property tells the interop layer to retrieve that immediately after calling this API so that .NET code can inspect it if neces-

sary. This API deals with strings, so interop needs to know which character representation is in use. It then uses `MarshalAs` on the two string arguments to tell the interop layer which of the many different string representations available in unmanaged code this particular API expects.

Prior to .NET 7, interop attributes were handled at runtime by the CLR (and that still happens if you use the older `DllImport` attribute instead of `LibraryImport`). However, that older approach relies on JIT compilation under the covers if any of the arguments or the return type need any kind of runtime processing, meaning it is unavailable if you're using Native AOT. The .NET SDK includes a code generator that looks for the `LibraryImport` attribute, and which generates code to convert between .NET and unmanaged types, enabling you to call into unmanaged APIs even when using Native AOT. (In fact, this generated code still ends up using the `DllImport` attribute, but it defines the target methods in such a way that the mappings between .NET types and unmanaged types are trivial, removing any need for the CLR to do any conversions.)

Defining and Consuming Attributes

The vast majority of attributes you will come across are not intrinsic to the runtime or compiler. They are defined by class libraries and have an effect only if you are using the relevant libraries or frameworks. You are free to do exactly the same in your own code—you can define your own attribute types. Because attributes don't do anything on their own—they don't even get instantiated unless something asks to see them—it is normally useful to define an attribute type only if you're writing some sort of framework, particularly one that is driven by reflection or compile-time code analysis.

For example, unit test frameworks often discover the test classes you write via reflection and enable you to control the test runner's behavior with attributes. Another example is how Visual Studio uses reflection to discover the properties of editable objects on design surfaces (such as UI controls), and it will look for certain attributes that enable you to customize the editing behavior. Another application of attributes is to opt out of rules applied by the static code analysis tools. (The .NET SDK has built-in tools for detecting potential problems in your code. This is an extensible

system, and NuGet packages can add analyzers that expand on this, potentially detecting common mistakes specific to a particular library.) Sometimes these tools make unhelpful suggestions, and you can suppress their warnings by annotating your code with attributes.

The common theme here is that some tool or framework examines your code and decides what to do based on what it finds. This is the kind of scenario in which attributes are a good fit. For example, attributes could be useful if you write an application that end users could extend. You might support loading of external assemblies that augment your application's behavior—this is often known as a *plug-in* model. It might be useful to define an attribute that allows a plug-in to provide descriptive information about itself. It's not strictly necessary to use attributes—you would probably define at least one interface that all plug-ins are required to implement, and you could have members in that interface for retrieving the necessary information. However, one advantage of using attributes is that you would not need to create an instance of the plug-in just to retrieve the description information. That would enable you to show the plug-in's details to the user before loading it, which might be important if constructing the plug-in could have side effects that the user might not want.

Attribute Types

[Example 14-23](#) shows how an attribute containing information about a plug-in might look.

Example 14-23. An attribute type

```
[AttributeUsage(AttributeTargets.Class)]
public class PluginInformationAttribute(string name, string author) : Attribute
{
    public string Name { get; } = name;

    public string Author { get; } = author;

    public string? Description { get; set; }
}
```

To act as an attribute, a type must derive from the `Attribute` base class. Although `Attribute` defines various static methods for discovering and retrieving attributes, it does not provide very much of interest for instances. We do not derive from it to get any particular functionality; we do so because the compiler will not let you use a type as an attribute unless it derives from `Attribute`.

Notice that my type's name ends in the word `Attribute`. This is not an absolute requirement, but it is an extremely widely used convention. As you saw earlier, it's even built into the compiler, which automatically adds the `Attribute` suffix if you leave it out when applying an attribute. So there's usually no reason not to follow this convention.

I've annotated my attribute type with an attribute. Most attribute types are annotated with the `AttributeUsageAttribute`, indicating the targets to which the attribute can usefully be applied. The C# compiler will enforce this. Since my attribute in [Example 14-23](#) states that it may be applied only to classes, the compiler will generate an error if you attempt to apply it to anything else.

NOTE

As you've seen, sometimes when we apply an attribute, we need to state its target. For example, an attribute appearing before a method targets that method unless you qualify it with the `return:` prefix. You might think you could leave out these prefixes with attributes that can target only certain members: if an attribute can be applied only to an assembly, do you really need the `assembly:` qualifier? However, C# doesn't let you leave it off. It uses the `AttributeUsageAttribute` only to verify that an attribute has not been misapplied.

My attribute defines only one constructor, so any code that uses it will have to pass the arguments that the constructor requires, as [Example 14-24](#) does.

Example 14-24. Applying an attribute

```
[PluginInformation("Reporting", "Endjin Ltd.")]
public class ReportingPlugin
{
```

```
    ...  
}
```

Attribute classes are free to define multiple constructor overloads to support different sets of information. They can also define properties as a way to support optional pieces of information. My attribute defines a `Description` property, which is not required because the constructor does not demand a value for it, but which I can set using the syntax I described earlier in this chapter. [Example 14-25](#) shows how that looks for my attribute.

Example 14-25. Providing an optional property value for an attribute

```
[PluginInformation("Reporting", "Endjin Ltd.",  
    Description = "Automated report generation")]  
public class ReportingPlugin  
{  
    ...  
}
```

So far, nothing I've shown will cause an instance of my `PluginInformationAttribute` type to be created. These annotations are simply instructions for how the attribute should be initialized if anything asks to see it. So, if this attribute is to be useful, I need to write some code that will look for it.

Retrieving Attributes

You can discover whether a particular kind of attribute has been applied using the reflection API, which can also instantiate the attribute for you. (You could also use Roslyn, the C# compiler API, but that's beyond the scope of this book.) In [Chapter 13](#), I showed all of the reflection types representing the various targets to which attributes can be applied—types such as `MethodInfo`, `Type`, and `PropertyInfo`. These all implement an interface called `ICustomAttributeProvider`, shown in [Example 14-26](#).

Example 14-26. ICustomAttributeProvider

```
public interface ICustomAttributeProvider
{
    object[] GetCustomAttributes(bool inherit);
    object[] GetCustomAttributes(Type attributeType, bool inherit);
    bool IsDefined(Type attributeType, bool inherit);
}
```

The `IsDefined` method simply tells you whether or not a particular attribute type is present—it does not instantiate it. The two `GetCustomAttributes` overloads create attributes and return them. (This is the point at which attributes are constructed and also when any properties the annotations specify are set.) The first overload returns all attributes applied to the target, while the second lets you request only those attributes of a particular type.

All of these methods take a `bool` argument that lets you specify whether you want only attributes that were applied directly to the target you're inspecting or also attributes applied to the base type or types.

This interface was introduced in .NET 1.0, so it does not use generics, meaning you need to cast the objects that come back. Fortunately, the `CustomAttributeExtensions` static class defines several extension methods that are more convenient to use. Instead of defining them for the `ICustomAttributeProvider` interface, it extends the reflection classes that offer attributes. For example, if you have a variable of type `Type`, you could call `GetCustomAttribute<PluginInformationAttribute>()` on it, which would construct and return the plug-in information attribute or return `null` if the attribute is not present. [Example 14-27](#) uses this to show all of the plug-in information from all the DLLs in a particular folder.

Example 14-27. Showing plug-in information

```
static void ShowPluginInformation(string pluginFolder)
{
    var dir = new DirectoryInfo(pluginFolder);
    foreach (FileInfo file in dir.GetFiles("*.dll"))
    {
```

```

        Assembly pluginAssembly = Assembly.LoadFrom(file.FullName);
        var plugins =
            from type in pluginAssembly.ExportedTypes
            let info = type.GetCustomAttribute<PluginInformationAttribute>()
            where info != null
            select new { type, info };

        foreach (var plugin in plugins)
        {
            Console.WriteLine($"Plugin type: {plugin.type.Name}");
            Console.WriteLine(
                $"Name: {plugin.info.Name}, written by {plugin.info.Author}");
            Console.WriteLine($"Description: {plugin.info.Description}");
        }
    }
}

```

There's one potential problem with this. I said that one benefit of attributes is that they can be retrieved without instantiating their target types. That's true here—I'm not constructing any of the plug-ins in [Example 14-27](#). However, I am loading the plug-in assemblies, and a possible side effect of enumerating the plug-ins would be to run static constructors in the plug-in DLLs. So, although I'm not deliberately running any code in those DLLs, I can't guarantee that no code from those DLLs will run. If my goal is to present a list of plug-ins to the user, and to load and run only the ones explicitly selected, I've failed, because I've given plug-in code a chance to run. However, we can fix this.

Metadata-Only Load

You do not need to load an assembly fully in order to retrieve attribute information. As I discussed in [Chapter 13](#), you can load an assembly for reflection purposes only with the `MetadataLoadContext` class. This prevents any of the code in the assembly from running but enables you to inspect the types it contains. However, this presents a challenge for attributes. The usual way to inspect an attribute's properties is to instantiate it by calling `GetCustomAttributes` or a related extension method. Since that involves constructing the attribute—which means running some code—it is not supported for assemblies loaded by `MetadataLoadContext` (not even if the attribute type in question were de-

finied in a different assembly that had been fully loaded in the normal way). If I modified [Example 14-27](#) to load the assembly with `MetadataLoadContext`, the call to `GetCustomAttribute<PluginInformationAttribute>` would throw an exception.

When loading for metadata only, you have to use the `GetCustomAttributesData` method. Instead of instantiating the attribute for you, this returns the information stored in the metadata—the instructions for creating the attribute. [Example 14-28](#) shows a version of the relevant code from [Example 14-27](#) modified to work this way. (It also includes the code required to initialize the `MetadataLoadContext`.)

Example 14-28. Retrieving attributes with the `MetadataLoadContext`

```
string[] runtimeAssemblies = Directory.GetFiles(
    RuntimeEnvironment.GetRuntimeDirectory(), "*.dll");
var paths = new List<string>(runtimeAssemblies);
paths.Add(file.FullName);

var resolver = new PathAssemblyResolver(paths);
var mlc = new MetadataLoadContext(resolver);

Assembly pluginAssembly = mlc.LoadFromAssemblyPath(file.FullName);
var plugins =
    from type in pluginAssembly.ExportedTypes
    let info = type.GetCustomAttributesData().SingleOrDefault(attrData =>
        attrData.AttributeType.FullName == pluginAttributeType.FullName)
    where info != null
    let description = info.NamedArguments
        .SingleOrDefault(a => a.MemberName == "Description")
    select new
    {
        type,
        Name = (string) info.ConstructorArguments[0].Value,
        Author = (string) info.ConstructorArguments[1].Value,
        Description =
            description == null ? null : description.TypedValue.Value
    };

foreach (var plugin in plugins)
{
```

```

        Console.WriteLine($"Plugin type: {plugin.type.Name}");
        Console.WriteLine($"Name: {plugin.Name}, written by {plugin.Author}");
        Console.WriteLine($"Description: {plugin.Description}");
    }

```

The code is rather more cumbersome because we don't get back an instance of the attribute. `GetCustomAttributesData` returns a collection of `CustomAttributeData` objects. [Example 14-28](#) uses LINQ's `SingleOrDefault` operator to find the entry for the `PluginInformationAttribute`, and if that's present, the `info` variable in the query will end up holding a reference to the relevant `CustomAttributeData` object. The code then picks through the constructor arguments and property values using the `ConstructorArguments` and `NamedArguments` properties, enabling it to retrieve the three descriptive text values embedded in the attribute.

As this demonstrates, the `MetadataLoadContext` adds complexity, so you should use it only if you need the benefits it offers. One benefit is the fact that it won't run any of the assemblies you load. It can also load assemblies that might be rejected if they were loaded normally (e.g., because they target a specific processor architecture that doesn't match your process). But if you don't need the metadata-only option, accessing the attributes directly, as [Example 14-27](#) does, is more convenient.

Generic Attribute Types

C# 11.0 and .NET 7.0 added support for defining generic attribute types. Showing such an attribute, and how to use it, is [Example 14-29](#).

Example 14-29. A generic attribute type

```

[AttributeUsage(AttributeTargets.Class)]
public class PluginHelpProviderAttribute<TProvider> : Attribute
    where TProvider : IPluginHelpProvider
{
    public Type ProviderType => typeof(TProvider);
}

// Usage:

```

```
[PluginInformation("Reporting", "Endjin Ltd.")]
[PluginHelpProviderAttribute<ReportingPluginHelpProvider>]
public class ReportingPlugin
{
    ...
}
```

Since this is a fairly recent feature, it is not yet widely used. Most attribute types that need to refer to a particular type (like the `ExpectedException` attribute shown in [Example 14-3](#)) just take a `Type` object as a constructor argument. That works perfectly well, but generic attribute types have one advantage: they can define constraints for their type arguments. [Example 14-29](#) requires that whatever type we plug in to this `PluginHelpProviderAttribute<TProvider>`, it must be a type that implements a specific interface, `IPluginHelpProvider`. If we try to use this attribute with a type that does not implement that interface, the compiler will report an error. With the old approach of passing a `Type` object to the attribute's constructor, you would either need to wait until the attribute is instantiated to detect the use of an inappropriate type, or write a code analyzer (which is a fairly complex task) to detect the problem at build time.

Summary

Attributes provide a way to embed custom data into an assembly's metadata. You can apply attributes to a type, any member of a type, a parameter, a return value, or even a whole assembly or one of its modules. A handful of attributes get special handling from the CLR, and a few control compiler features, but most have no intrinsic behavior, acting merely as passive information containers. Attributes do not even get instantiated unless something asks to see them. All of this makes attributes most useful in systems with reflection-driven behavior—if you already have one of the reflection API objects such as `ParameterInfo` or `Type`, you can ask it directly for attributes. You therefore most often see attributes used in frameworks that inspect your code with reflection, such as unit test frameworks, serialization frameworks, data-driven UI elements like Visual Studio's Properties panel, or plug-in frameworks. Sometimes frameworks of this kind can use the C# compiler API (Roslyn) to discover

attributes at build time. This enables them to avoid relying on reflection, making them compatible with Native AOT. If you are using a framework of this kind, you will typically be able to configure its behavior by annotating your code with the attributes the framework recognizes. If you are writing this sort of framework, then it may make sense to define your own attribute types.